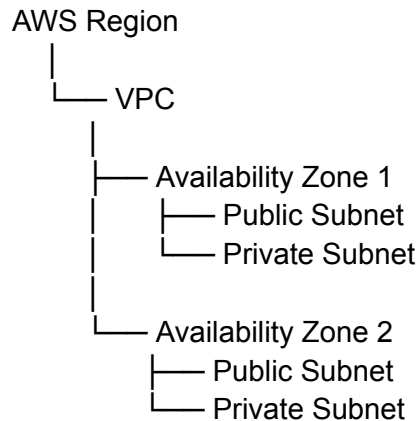


AWS Networking Fundamentals

The big picture you should understand today is:



This is the foundation for things like EC2, RDS, Load Balancers, Kubernetes, Docker deployments, and eventually CI/CD infrastructure.

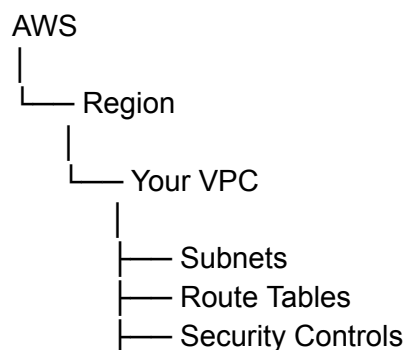
1. What is a VPC?

VPC = Virtual Private Cloud

A VPC is a **logically isolated virtual network that you create inside AWS**.

Think of it as creating your own private networking environment inside AWS.

For example:



You decide things like:

- IP address range
 - Subnets
 - Routing
 - Network access
 - Security rules
 - Internet connectivity
 - Communication between resources
-

2. VPC as a Virtual Data Center

A useful beginner analogy is:

VPC $\approx$ your own logical data-center network in AWS.

But don't take "virtual data center" too literally.

A VPC isn't literally a physical data center.

It's a **software-defined/logical networking boundary** within AWS.

So if someone asks:

"Is a VPC a physical network?"

Answer:

No. It's a logical virtual network implemented within AWS's physical networking infrastructure.

3. Example VPC

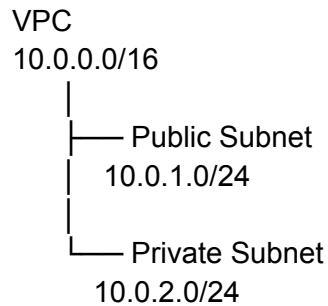
Suppose we create:

VPC CIDR:

10.0.0.0/16

This defines the IP address space available to the VPC.

We can then divide that address space into smaller networks:



This brings us to an important networking concept:

CIDR

You will see CIDR **everywhere** in AWS.

CIDR stands for:

Classless Inter-Domain Routing

Examples:

10.0.0.0/16
10.0.1.0/24
192.168.1.0/24

The **/16** or **/24** tells us how much of the IP address represents the network portion.

For now, don't get stuck doing binary calculations.

Just understand:

10.0.0.0/16
↓
Large network

10.0.1.0/24

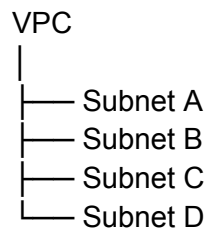
↓
Smaller network

Later, when we do networking properly, we'll cover CIDR calculation, subnetting, private IP ranges, routing, NAT, etc.

4. What is a Subnet?

A subnet is a smaller IP network inside your VPC.

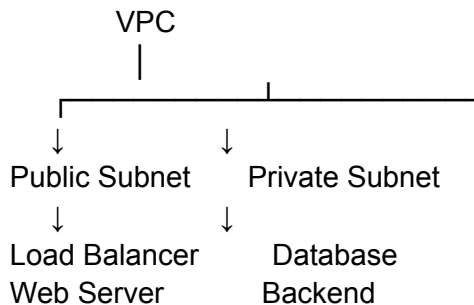
Think:



Why divide a VPC into subnets?

Because different resources have different networking requirements.

For example:



This lets us separate resources logically.

5. VPC vs Subnet

This is something you should be able to answer instantly.

VPC

The **larger network**.

Subnet

A **smaller network inside the VPC**.

Think:

Country



State



City

Not a perfect technical analogy, but useful mentally:

VPC



Subnet



Resources

6. Very Important: Availability Zone and Subnet

You mentioned:

Each subnet belongs to one Availability Zone only.

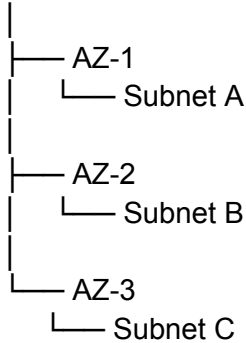
 **Correct.**

An AWS subnet is associated with **one Availability Zone**.

It cannot span multiple AZs.

For example:

Region: Mumbai



You cannot create:

✗ One subnet spanning AZ-1 + AZ-2

Instead:

AZ-1 → Subnet A

AZ-2 → Subnet B

This becomes extremely important when designing highly available architectures.

7. Public Subnet

Now we get into the fun stuff.

A **public subnet** is a subnet whose routing configuration provides a path to an **Internet Gateway**.

This is the technically important definition.

Don't memorize:

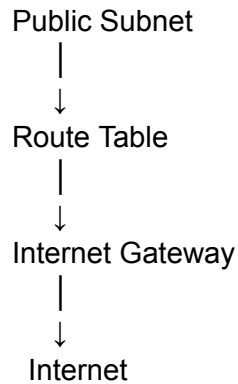
"Public subnet = subnet with internet."

That's slightly too simplistic.

The better definition is:

A subnet is considered public when its route table has a route to an Internet Gateway.

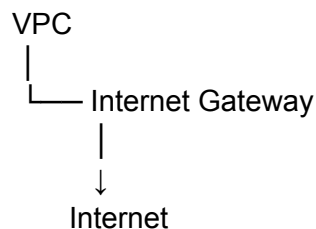
For example:



8. What is an Internet Gateway?

Internet Gateway (IGW) is an AWS VPC component that enables communication between resources in a VPC and the internet, subject to routing and security configuration.

Think:



But here's an important thing:

Having an Internet Gateway does NOT automatically make every resource public.

You also need appropriate:

- route table
- IP addressing

- security rules
- resource configuration

So:

Internet Gateway $\neq$ automatic internet access

This distinction is important.

9. What Goes in a Public Subnet?

Typical resources include:

Load Balancers

For example:

Internet
↓
Application Load Balancer
↓
Private application servers

Web-facing infrastructure

Depending on architecture, web servers or bastion hosts can exist in public subnets.

But modern architectures often minimize direct exposure.

10. Private Subnet

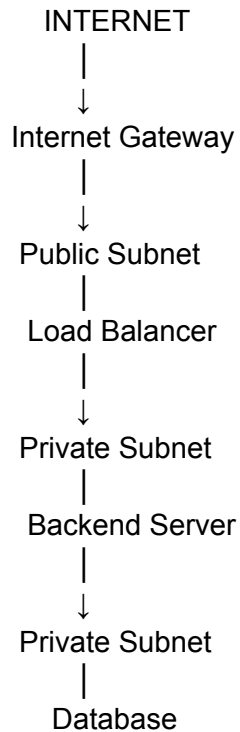
A private subnet doesn't have a direct route to an Internet Gateway.

Typical resources:

- Databases
- Backend servers

- Internal applications
- Internal services

Example:



This is a very common architecture.

11. Does Private Subnet Mean "No Internet Ever"?

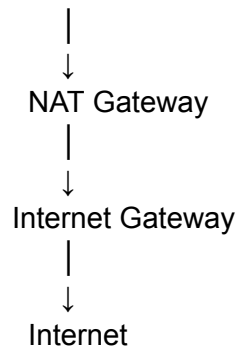
Not necessarily.

This is another important correction.

A resource in a private subnet can potentially access the internet **outbound** through a NAT Gateway, without being directly reachable from the internet.

For example:

Private Server



This is useful when a backend server needs to:

- download packages
- access external APIs
- retrieve updates

while still not accepting unsolicited inbound internet connections.

We'll study **NAT Gateway** properly later.

12. Public vs Private Subnet

Feature	Public Subnet	Private Subnet
Route to Internet Gateway	Yes	No
Direct inbound internet access	Possible*	Not directly
Common usage	Load balancers, web-facing resources	Backend, databases
Internet-facing	Can be	Normally no
Security exposure	Higher	Lower
Typical architecture	Front door	Internal layer

*Assuming the resource itself has appropriate addressing and security configuration.

13. "Databases Should Always Be Private"

Your notes say:

Databases should always be in private subnets.

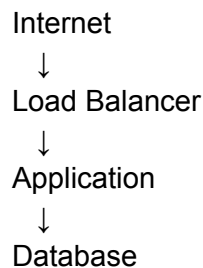
As a **security best practice**, yes — that's the mindset you should develop.

But don't turn "always" into a universal law.

The better statement is:

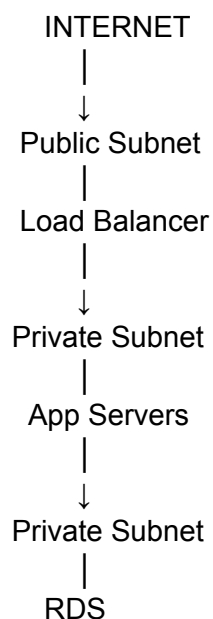
Databases should generally not be directly exposed to the public internet.

For a normal production architecture:



The database is internal.

For example:



This follows the **principle of least exposure**.

14. Why Separate Public and Private Subnets?

Imagine you have:

Frontend
Backend
Database

You don't want:

Internet
↓
Database 🤔

Instead:

Internet
↓
Frontend / Load Balancer
↓
Backend
↓
Database

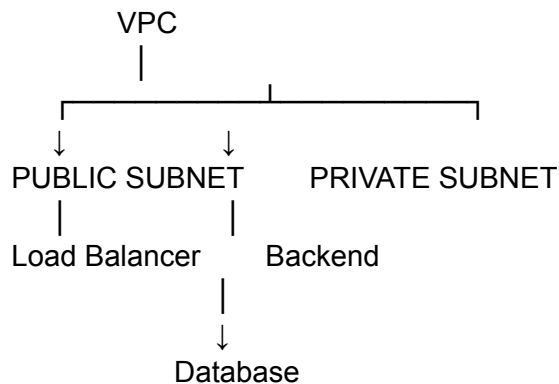
Each layer has a different level of exposure.

This is basically **network segmentation**.

15. Security Through Network Segmentation

One of the biggest reasons we use subnets is to separate different workloads.

For example:



The database doesn't need to be accessible from everywhere.

Ideally:

Internet  → Database
Backend  → Database

This dramatically reduces the attack surface.

16. But Subnets Alone Don't Provide Security

This is **VERY IMPORTANT**.

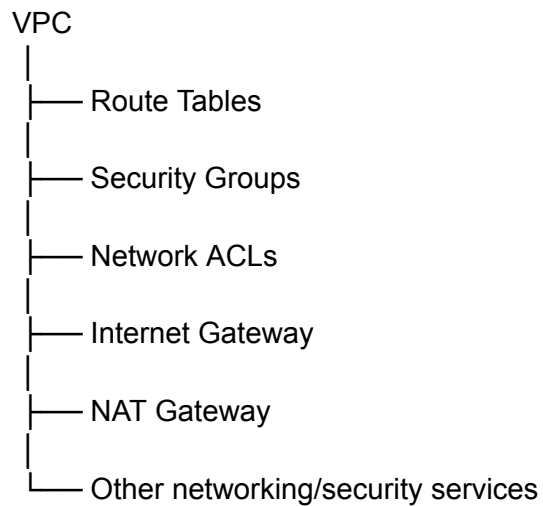
Some beginners think:

"Private subnet = secure."

Not exactly.

Security comes from multiple layers.

AWS networking security can involve:



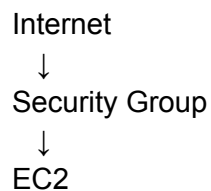
We'll study these individually.

17. Security Groups

You'll encounter this very soon.

A **Security Group (SG)** acts as a virtual firewall associated with resources such as EC2 instances.

Imagine:



You can define rules like:

Allow TCP 443

Allow TCP 22 from trusted source

And restrict unwanted traffic.

Security Groups are **stateful**.

We'll cover exactly what that means later.

18. Network ACL

Another AWS networking security mechanism is:

NACL = Network Access Control List

It operates at the subnet level.

Conceptually:

Subnet
↓
NACL
↓
Resources

Compared with:

Security Group
↓
Resource/Network interface

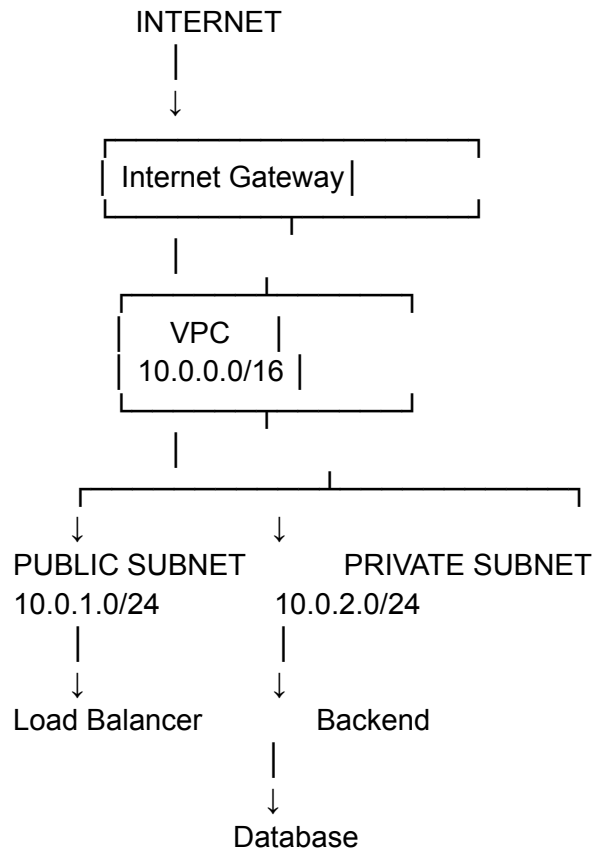
A major difference you'll eventually need to remember:

	Security Group	NACL
Associated with	Resource/network interface	Subnet
Stateful	Yes	No
Rules	Allow	Allow + Deny
Purpose	Resource-level traffic control	Subnet-level traffic control

Don't worry if this feels confusing now. We'll drill it when we get to security groups.

19. Full VPC Architecture

Here's the architecture I want you to visualize:



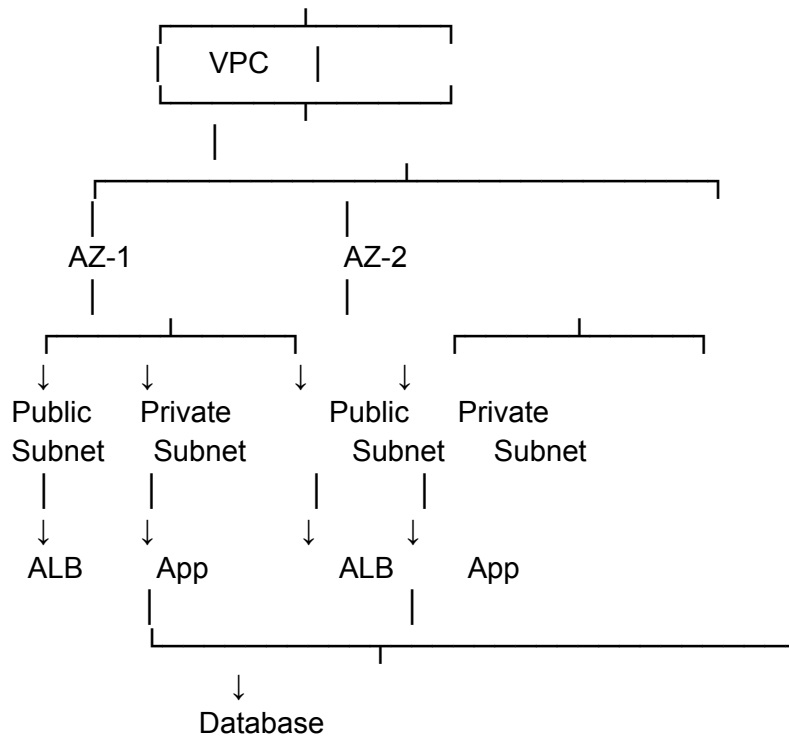
That is the basic mental model.

20. Multi-AZ VPC Architecture

Now combine today's topic with yesterday's Availability Zone lesson.

A real production architecture might look like:





Now we're combining:

Region → AZ → VPC → Subnet → Resources

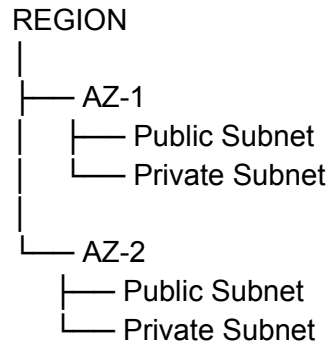
This is the cloud architecture mindset you want.

21. The Most Important Mental Model

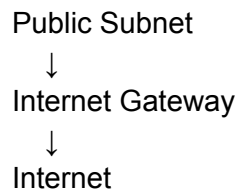
Memorize this:

REGION
↓
VPC
↓
SUBNET
↓
RESOURCE

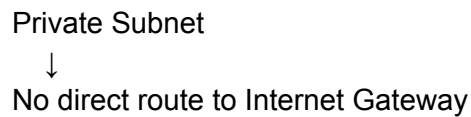
But because subnets are AZ-specific:



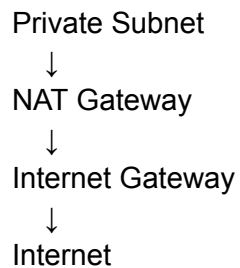
Then:



while:



and potentially:



for outbound internet access.

Your original note says:

"Without a VPC, cloud resources cannot communicate securely."

Don't memorize that as a universal fact.

A better statement is:

In AWS, a VPC provides the core logical networking environment in which many AWS resources operate, allowing you to control IP addressing, routing, and network access.

Also, not every AWS service is simply "inside your VPC" in the same way. Some AWS services are regional/global managed services with different networking models.

That's an important distinction as you move into advanced AWS.

Questions

Try answering these yourself before checking the answers.

Q1. What is a VPC?

Answer: A logically isolated virtual network in AWS where you can launch and connect resources while controlling IP addressing, routing and network access.

Q2. What is a subnet?

Answer: A subnet is a smaller IP network within a VPC. In AWS, each subnet is associated with a single Availability Zone.

Q3. What makes a subnet public?

Answer: Its route table has a route to an Internet Gateway.

Q4. Is a public subnet automatically accessible from the internet?

Answer: No. Appropriate routing, IP addressing, security-group/NACL rules and resource configuration are also required.

Q5. Can a subnet span multiple AZs?

Answer: No. A subnet belongs to one AZ.

Q6. Can a VPC span multiple AZs?

Answer: Yes.

Q7. Can a VPC span multiple Regions?

Answer: No. A VPC is associated with a single AWS Region.

This one is **very important**:

One VPC



One Region



Multiple AZs possible

Q8. Where should a database normally be placed?

Answer: In a private subnet, without direct public internet exposure.

Q9. Can a private subnet access the internet?

Answer: Yes, potentially through a NAT Gateway for outbound connectivity, while remaining without a direct route from the subnet to an Internet Gateway.

Q10. What is the purpose of a VPC?

Think:

Isolation + IP addressing + routing + network control.

Trap Questions

"Is a VPC the same as a subnet?"

✗ No.

VPC



Subnet

"Can a subnet exist in multiple Availability Zones?"

✗ No.

One subnet → **one AZ**.

"Does public subnet mean every EC2 inside it is publicly accessible?"

✗ No.

The EC2 also needs appropriate network configuration, including public addressing where applicable and security rules.

"Can a private subnet access the internet?"

✓ Yes, for outbound access through NAT, if configured appropriately.

"Does private subnet mean completely isolated from the internet?"

Not necessarily.

It means **no direct route to an Internet Gateway**.

"Is VPC security handled only by Security Groups?"

✗ No.

You have multiple networking/security mechanisms, including:

- Security Groups
 - NACLs
 - Route tables
 - Internet Gateway
 - NAT Gateway
 - IAM and other AWS security controls
-



Revision Notes

VPC



Virtual Private Cloud



Logical isolated network in AWS

CIDR



Defines IP address range

SUBNET



Smaller network inside VPC



One subnet = one AZ

PUBLIC SUBNET



Route to Internet Gateway



Can support internet-facing resources

PRIVATE SUBNET



No direct route to Internet Gateway



Backend / databases / internal services

INTERNET GATEWAY



Connects VPC to internet

NAT GATEWAY



Allows private resources to initiate outbound internet connections

SECURITY GROUP



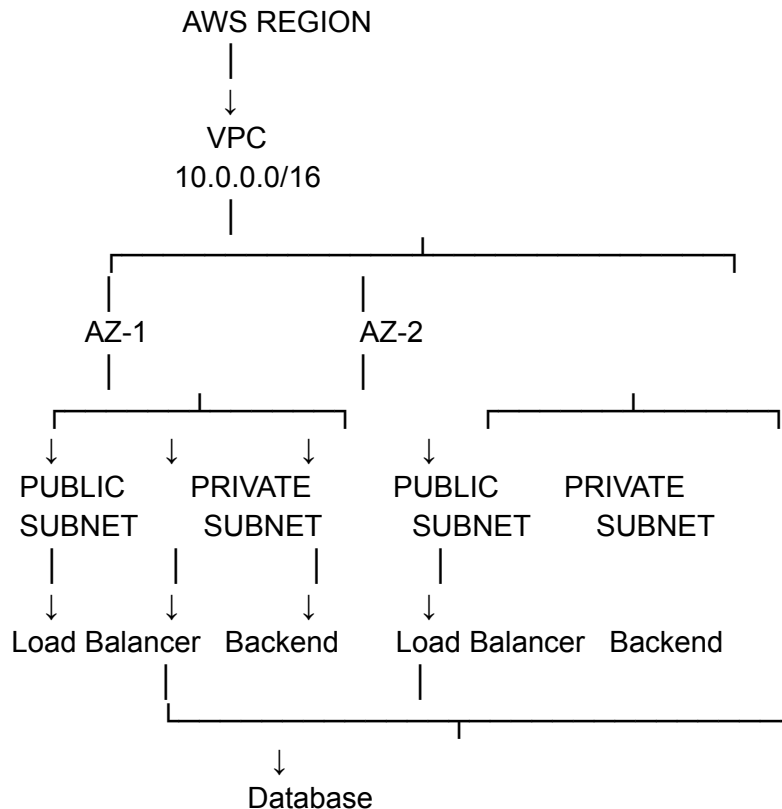
Stateful resource-level firewall

NACL



Stateless subnet-level traffic control

🔥 The architecture you should now see in your head



If you understand **this diagram**, you're genuinely starting to understand AWS networking rather than just memorizing AWS vocabulary.

And one thing: **don't rush past networking**. For your DevOps + multi-cloud path, networking is one of those boring-looking topics that later becomes *absolutely everywhere* — Docker networking, Kubernetes Services/Ingress, AWS VPC, Azure VNet, GCP VPC, CI/CD infrastructure, microservices, security, load balancing, etc. This is worth getting rock solid.